

VGGNet - Backbone Paper Review

홍창희, 박민선

July. 11. 2025

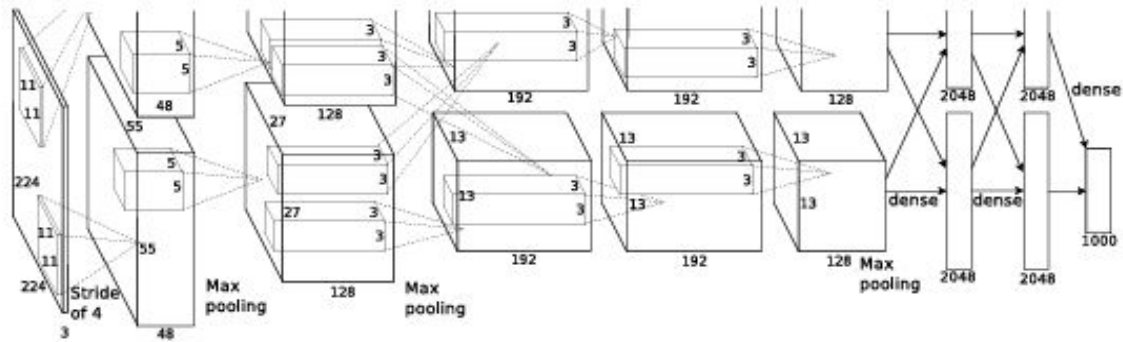
Medical AI Research Center

■ VGGNet 백본 논문

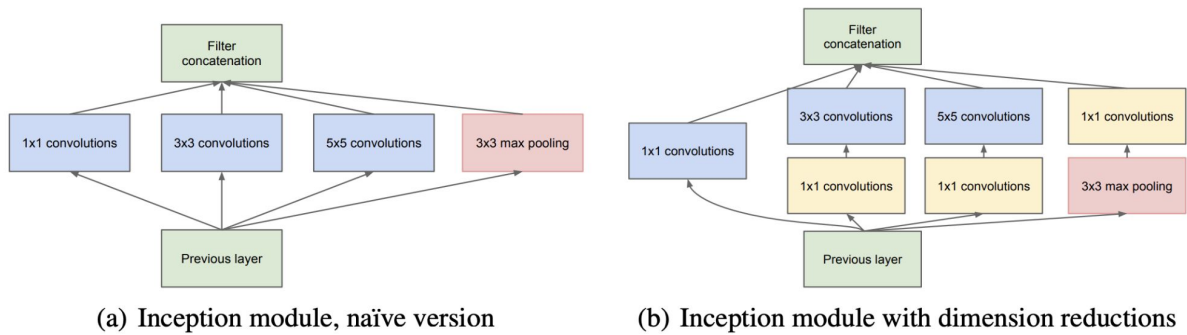
1. Introduction
 2. Architecture
 3. Train & Test
 4. Experiments
 5. Conclusion
- +) 논문 구현 실습

■ VGGNet 활용 임상 저널

1. 소개
2. 사용 데이터셋
3. 모델 적용 & 성능



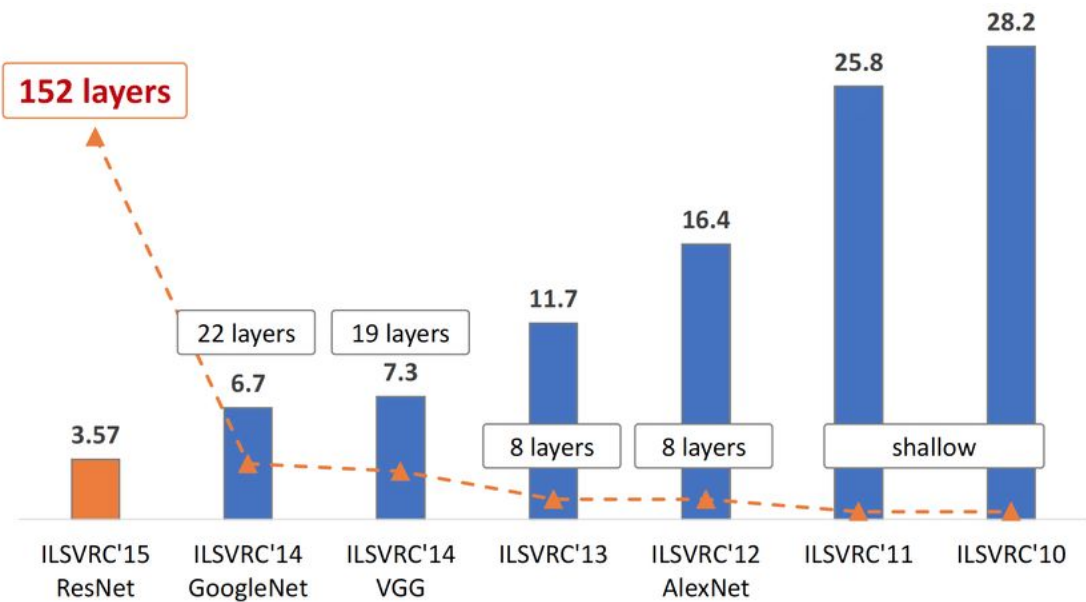
ex) ALEXNET



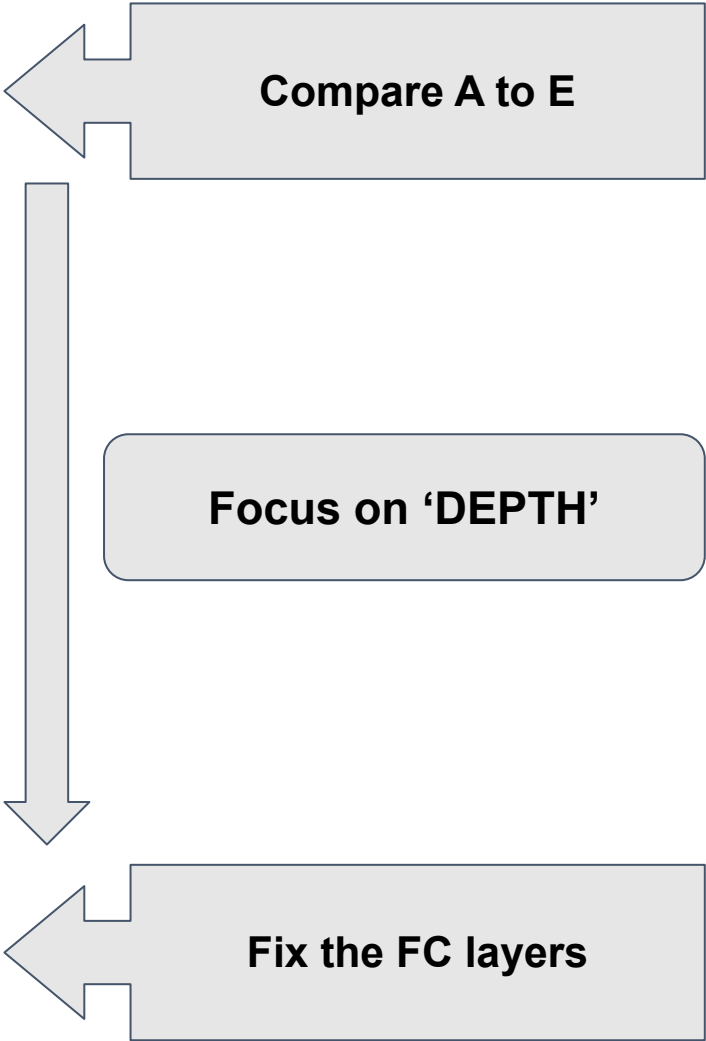
(a) Inception module, naïve version

(b) Inception module with dimension reductions

Figure 2: Inception module

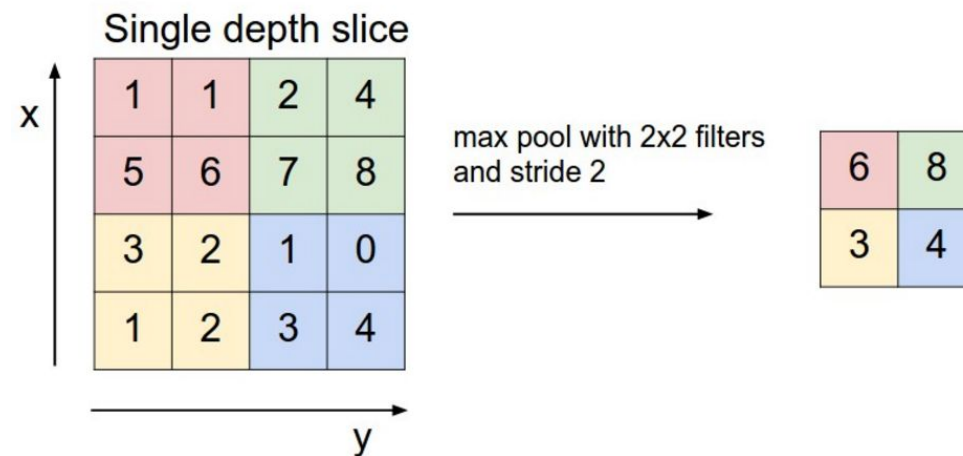
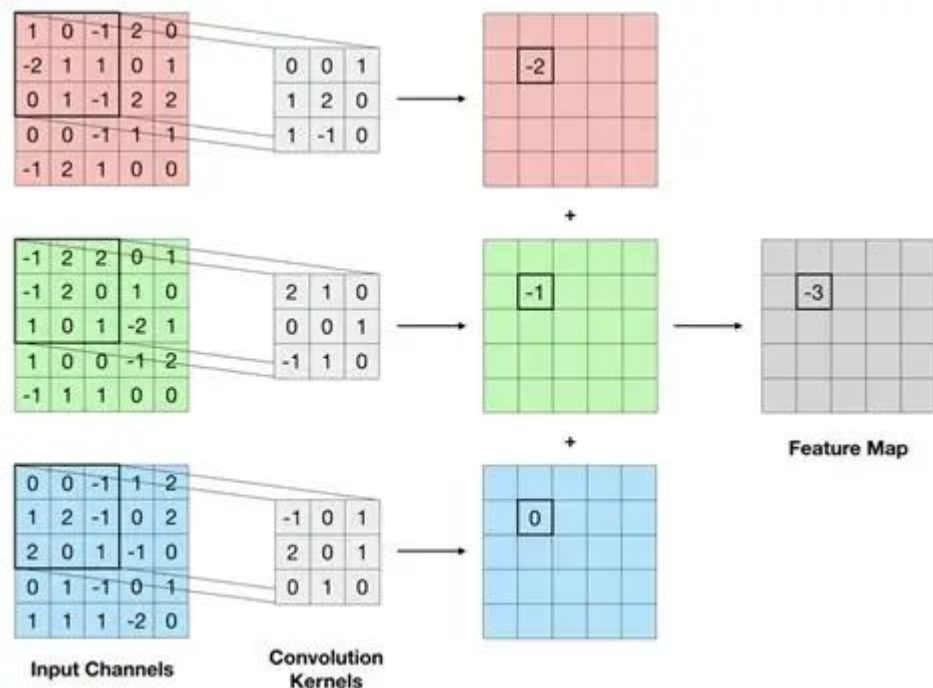


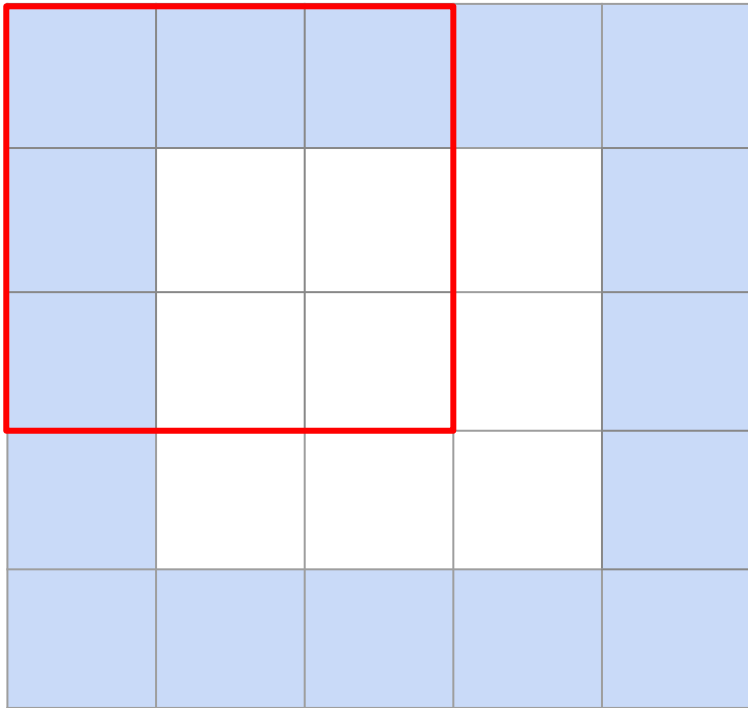
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					



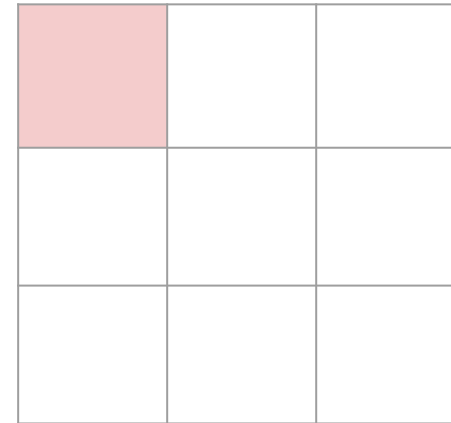
Convolution(합성곱): 입력 이미지에서 에지(윤곽선)나 패턴과 같은 중요한 특징을 추출한다. 학습 가능한 필터를 사용하여 특징 맵(feature map)을 생성한다.

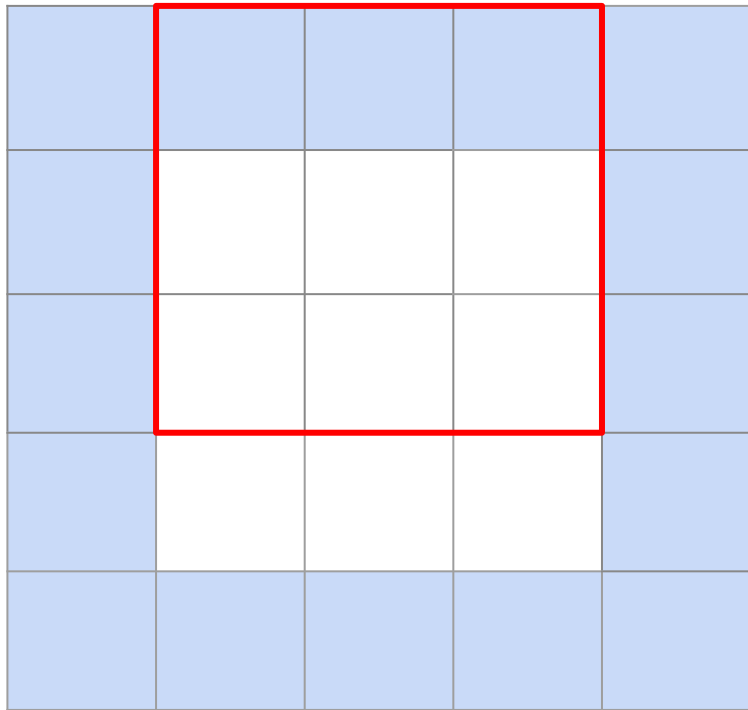
Max Pooling(최대 풀링): 가장 중요한 정보만 남기고 공간적인 크기를 줄인다. 이를 통해 네트워크가 위치 변화에 더 robust해지고 연산량도 줄일 수 있다.



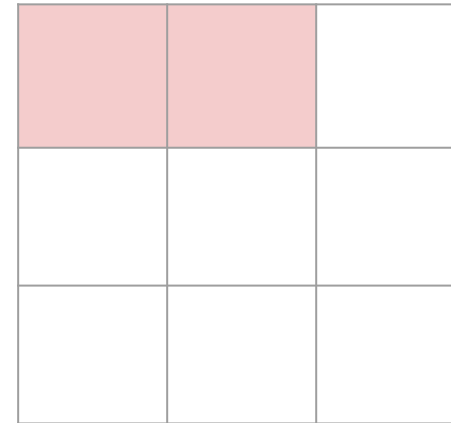


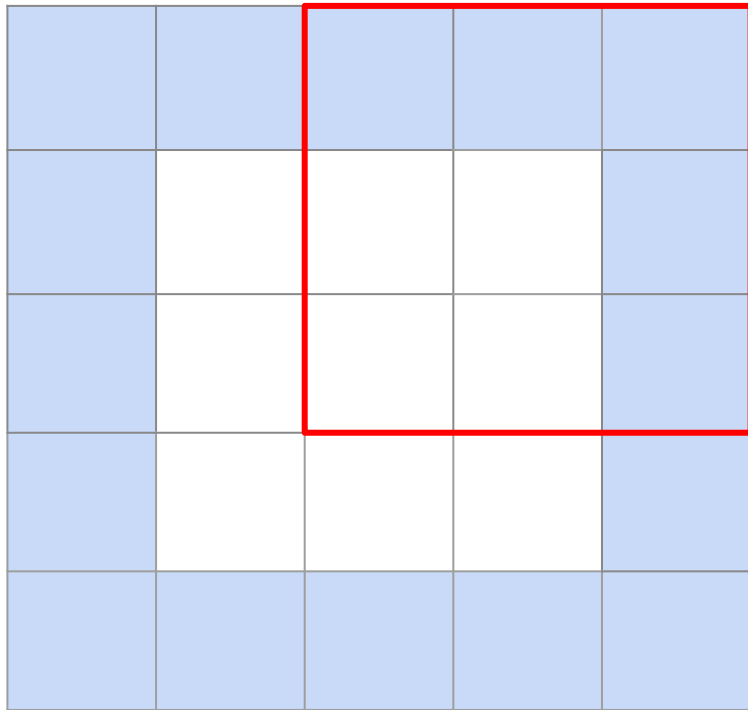
3x3 filter, stride=1, padding=1



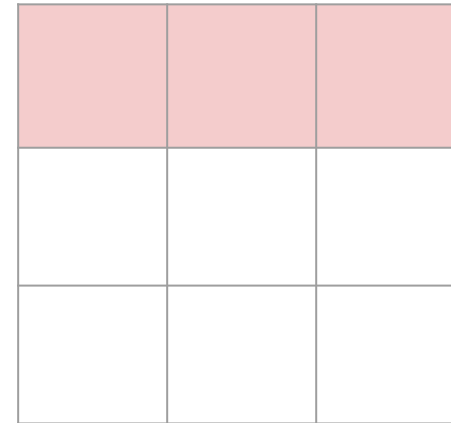


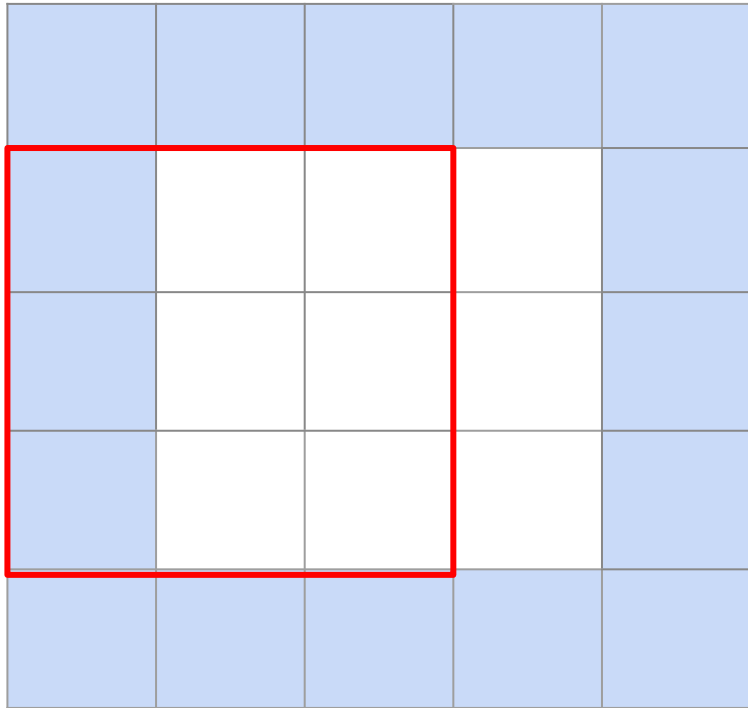
3x3 filter, stride=1, padding=1



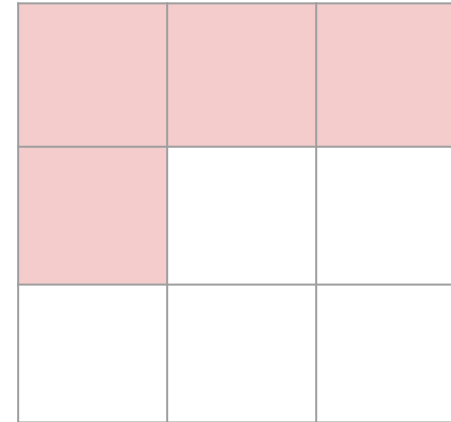


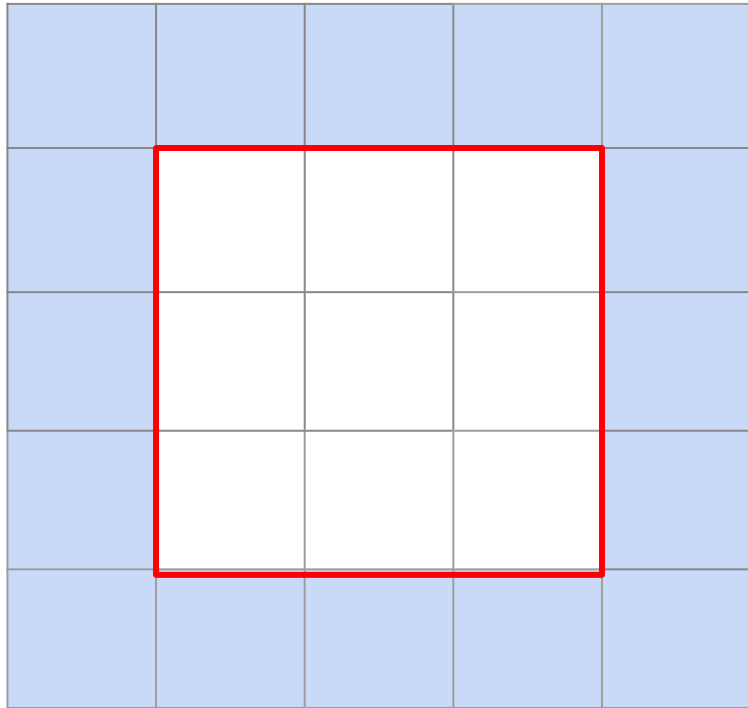
3x3 filter, stride=1, padding=1



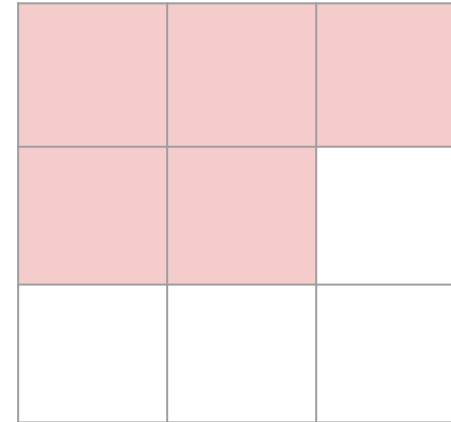


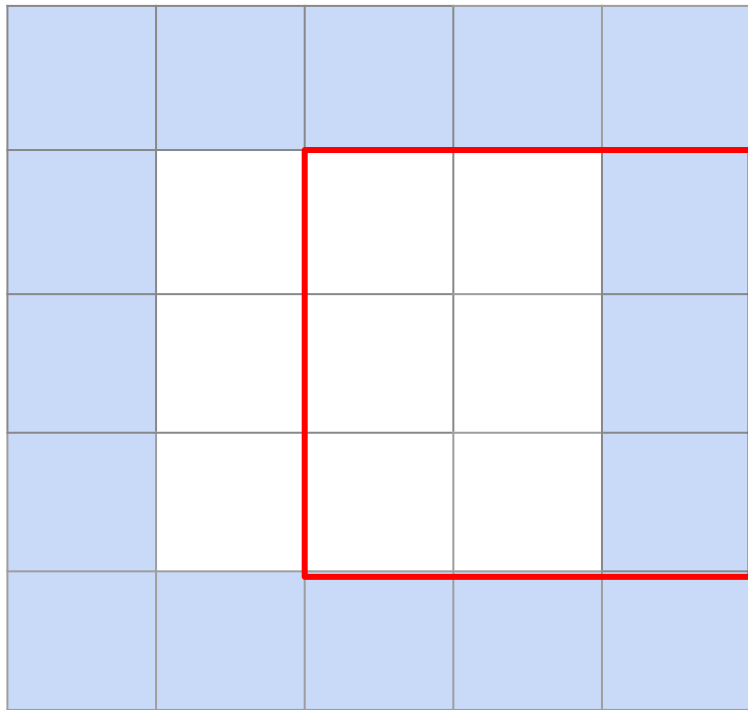
3x3 filter, stride=1, padding=1



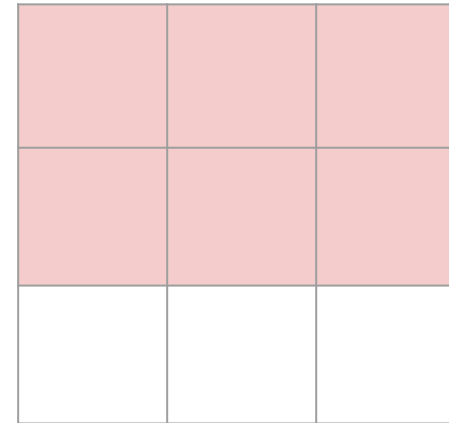


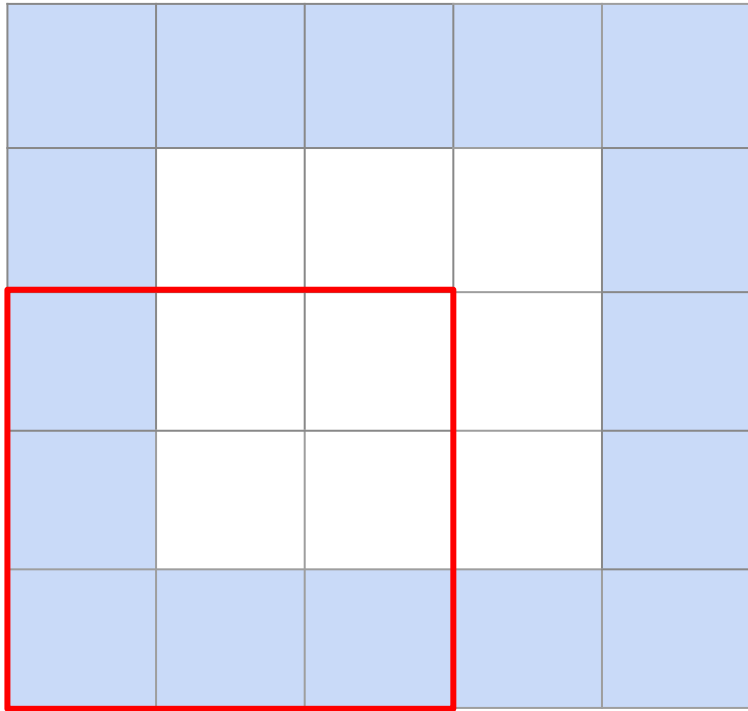
3x3 filter, stride=1, padding=1



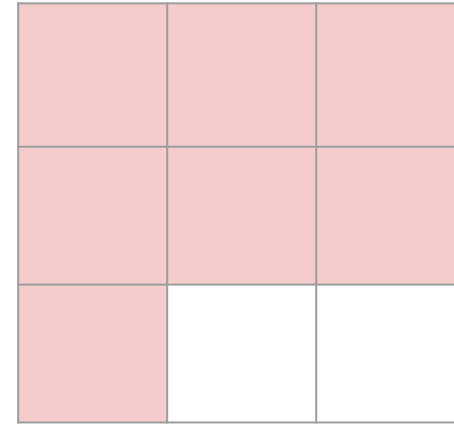


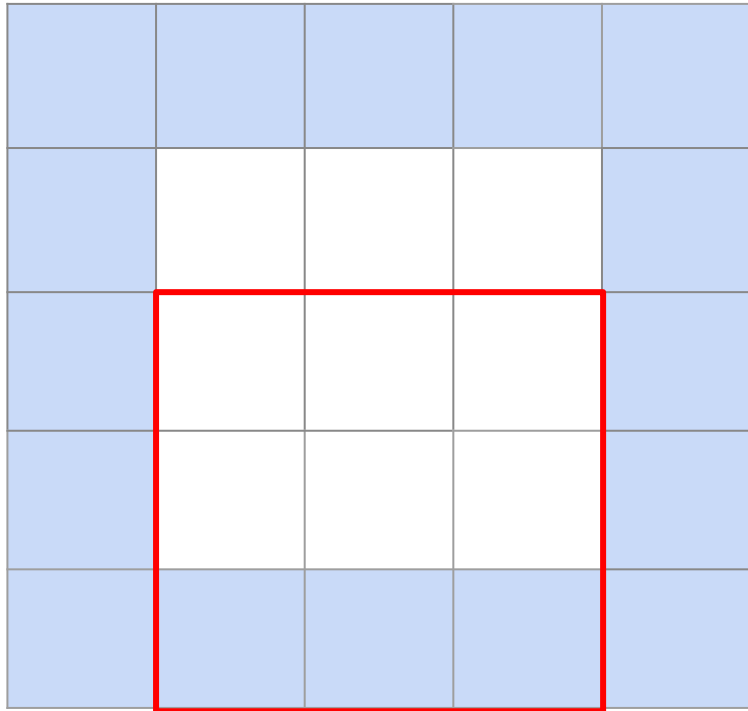
3x3 filter, stride=1, padding=1



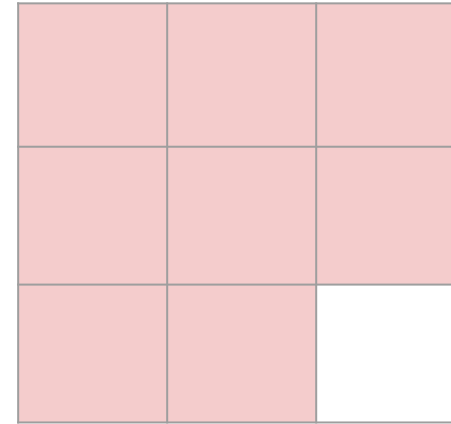


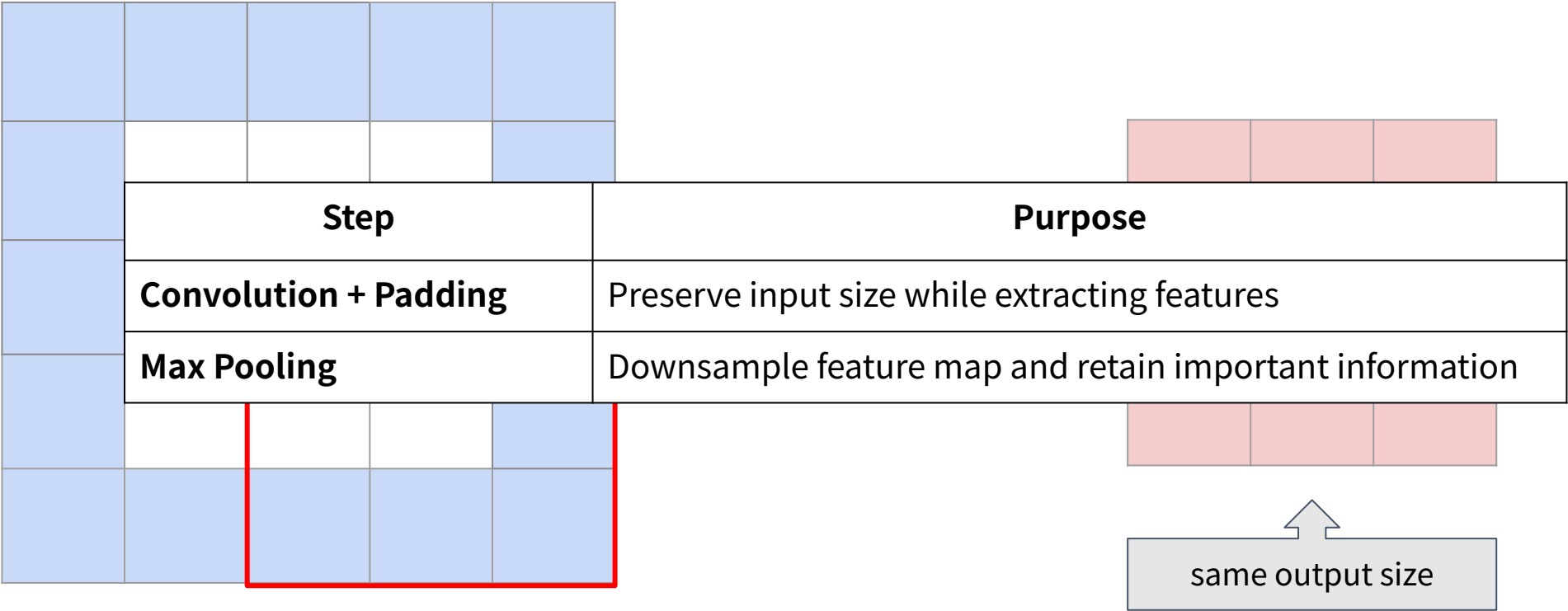
3x3 filter, stride=1, padding=1

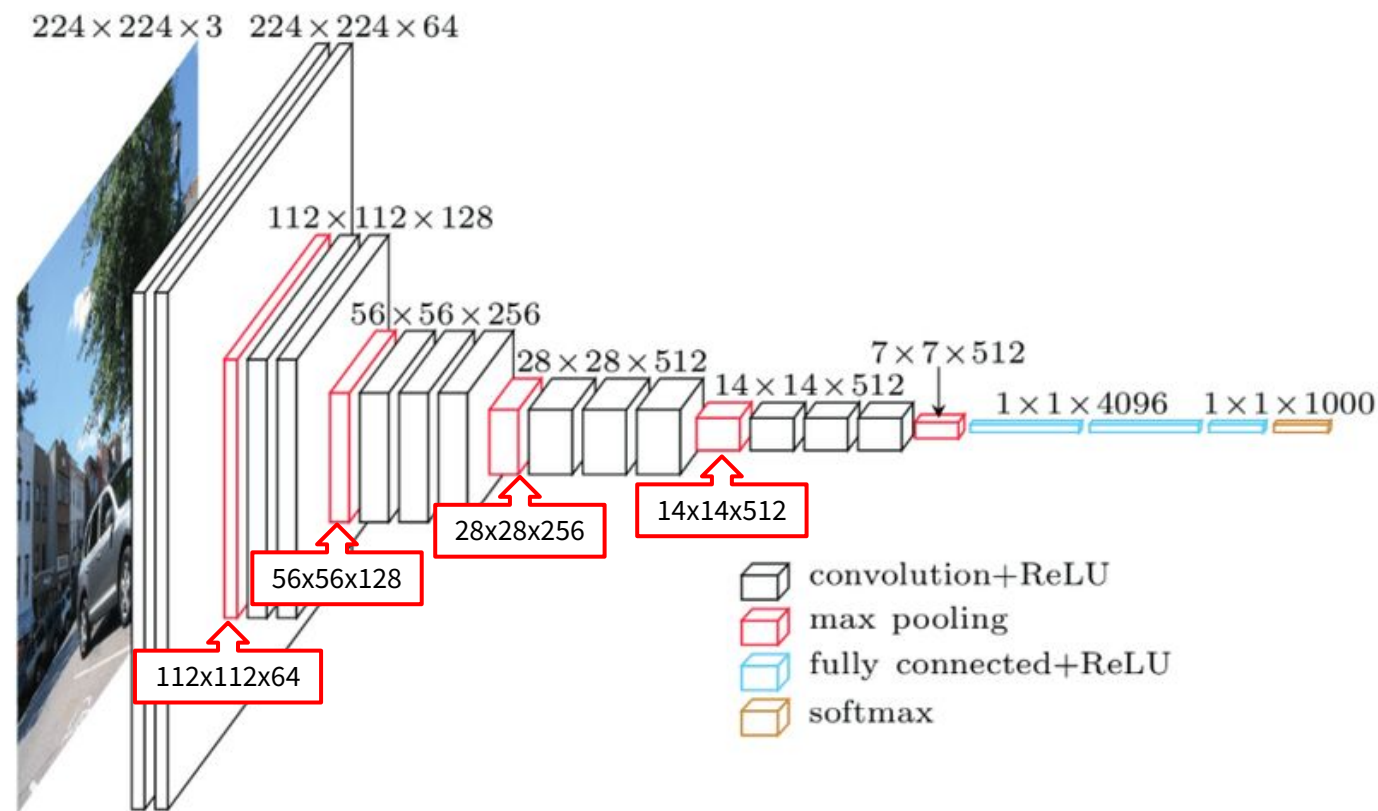




3x3 filter, stride=1, padding=1







ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64	conv3-64	conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128	conv3-128	conv3-128
maxpool					
conv3-256	conv3-256	conv3-256 conv3-256	conv3-256	conv3-256	conv3-256
conv3-256	conv3-256	conv3-256	conv3-256 conv1-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
maxpool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512 conv1-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
conv3-512	conv3-512	conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512 conv1-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

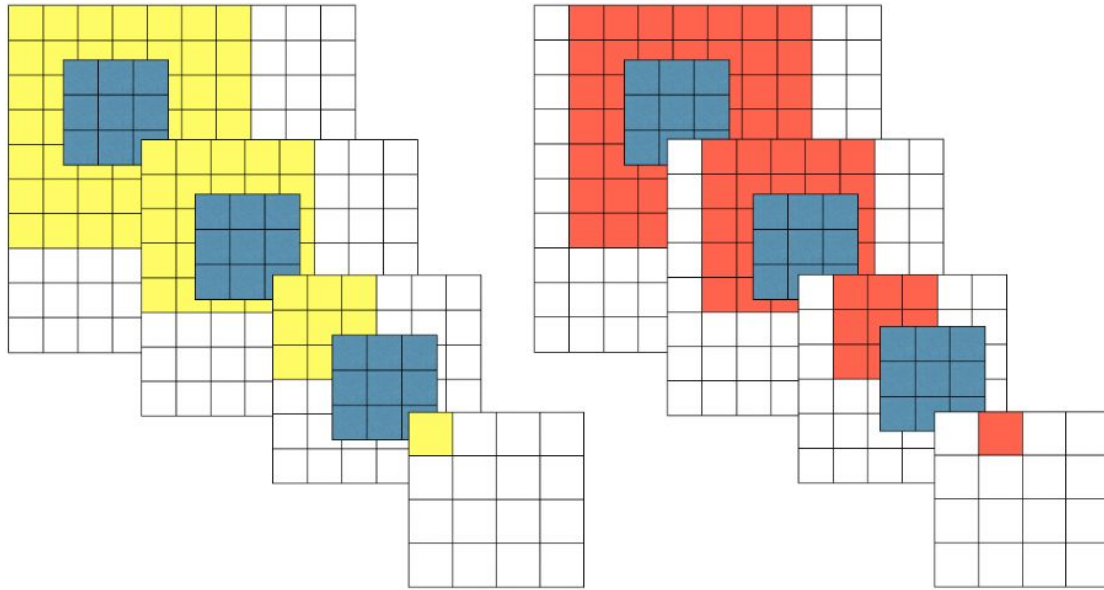
Benefit of using 3×3 filter

1. Decreasing number of parameters
 - using 3×3 filter = $3 \times 3 \times 3 \times C \times C$
 - using 7×7 filter = $7 \times 7 \times C \times C$

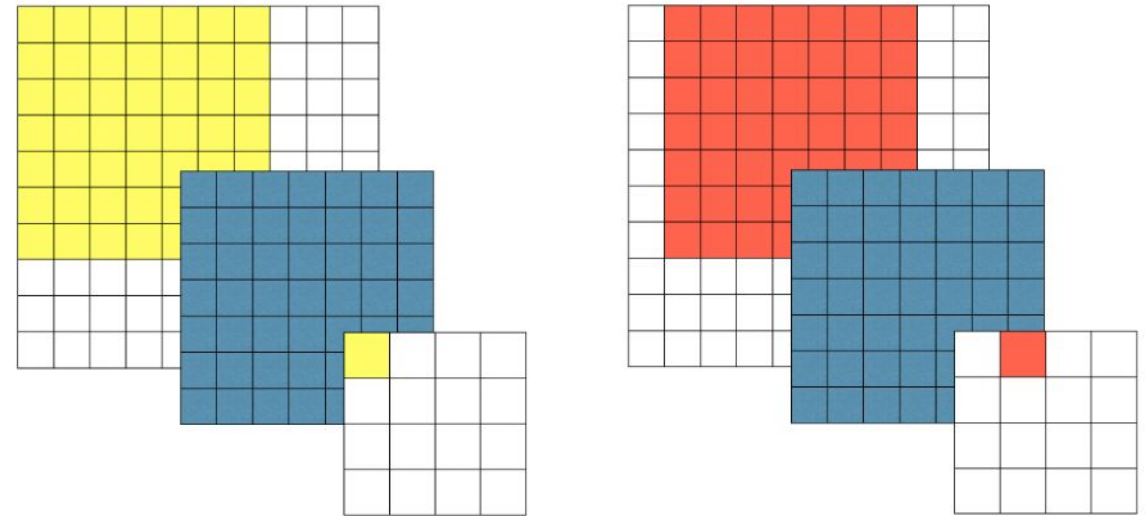
C: number of channels

2. Increases non-linearity

→ more efficient computation and better performance



using 3×3 filter



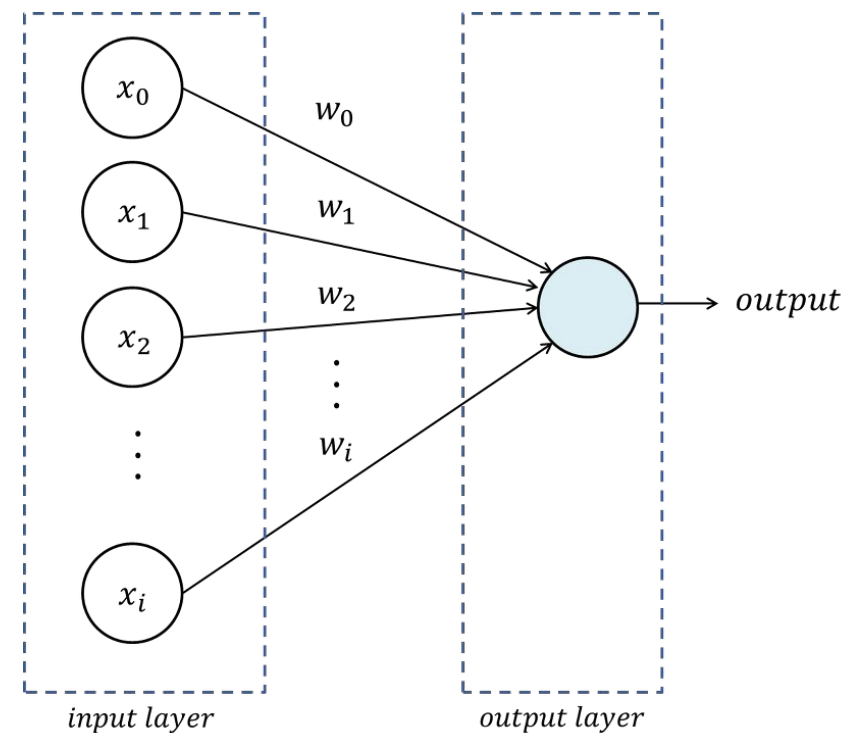
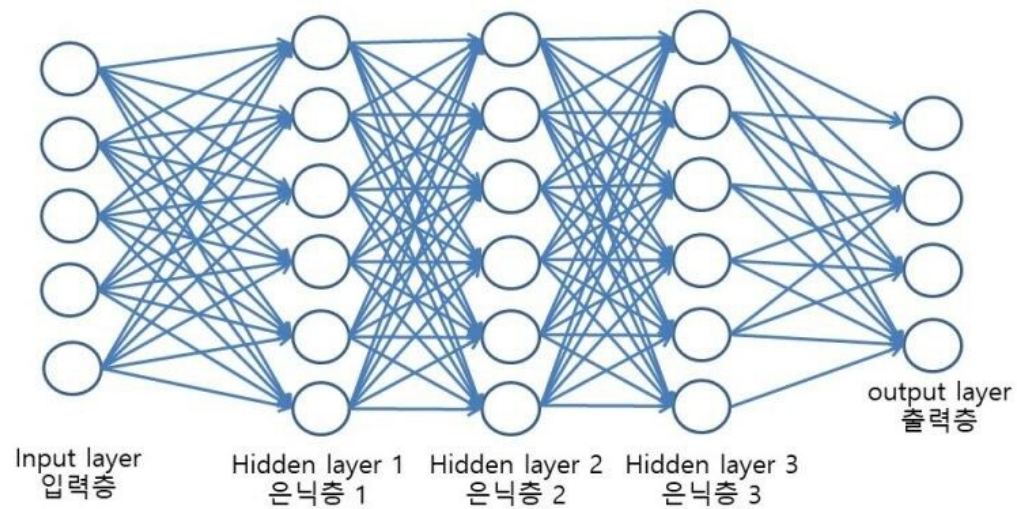
using 7×7 filter

Benefit of using 3×3 filter

1. Decreasing number of parameters
 - using 3×3 filter = $3 \times 3 \times 3 \times C \times C$
 - using 7×7 filter = $7 \times 7 \times C \times C$

2. Increases non-linearity

→ more efficient computation and better performance



[Optimization]

- 다중 분류 (Softmax + Cross Entropy)
- Mini-batch gradient descent (256)
- Momentum(0.9)
- weight decay (L2 Norm)
- 처음 2개의 FC레이어에 dropout(0.5)
- 초기 learning rate: 0.01

[Parameter Initialization]

- 깊은 모델들은 A 모델의 파라미터를 가져와
parameter initialization(첫 4개 conv층과 3개의 FC층만)
(나머지는 평균=0, 분산=0.01인 정규분포에서 랜덤 초기화)

parameters

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

[Single/Multi-Scale]

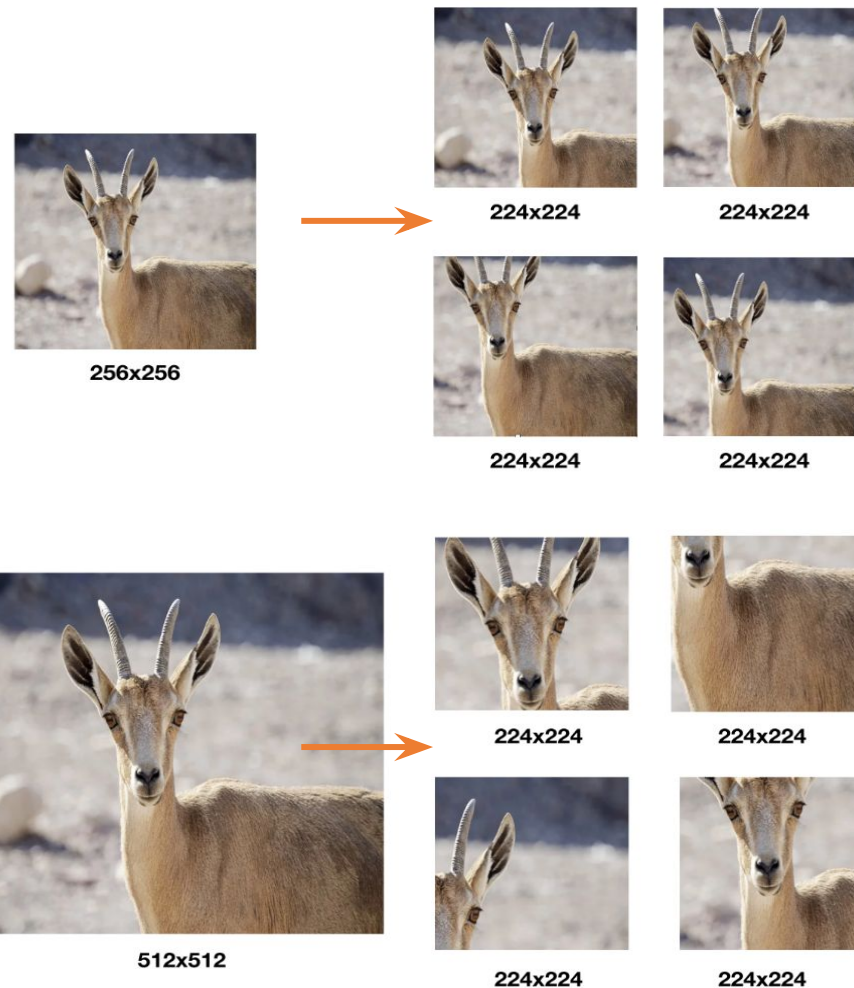
- **Single-scale**

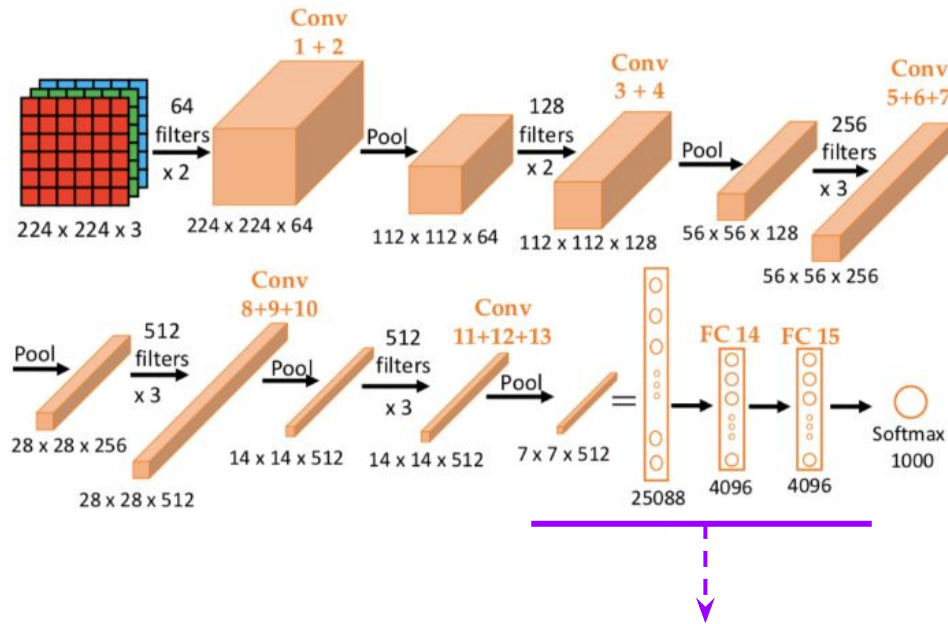
입력 이미지의 S 를 $S = 256$, 또는 $S = 384$ 로
scale을 고정하는 방식

- **Multi-scale**

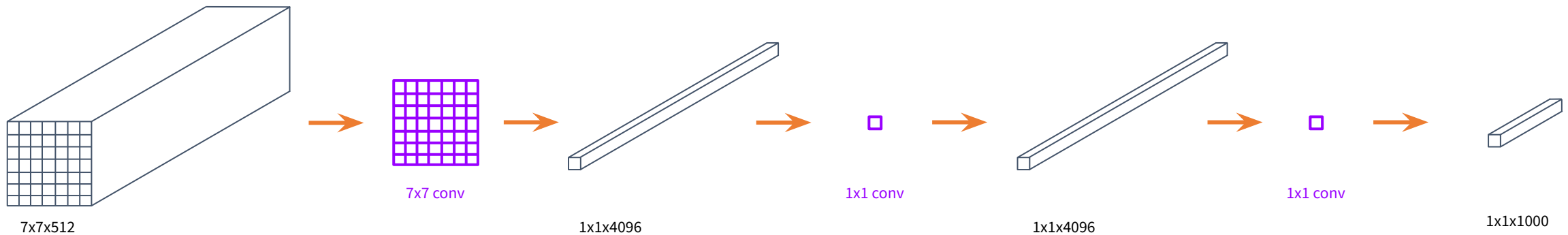
S 를 S_{min} 과 S_{max} 범위에서 무작위로 선택하는 방식
($S \in [256, 512]$)
다양한 크기에 대한 대응이 가능하여 정확도가 올라감
이를 **Scale Jittering**이라고 함

- > 학습 이미지를 임의의 크기 스케일 S 에서
여러개의 **224x224** 이미지들로 **크롭**한뒤 학습
(+ 랜덤 horizontal flip)





- Training 완료된 모델을 Testing할 때는 신경망의 마지막 3개의 **Fully-Connected layer**를 **Convolutional layer**로 변환하여 사용
- 첫 번째 Fully-Connected layer는 **7x7 Conv**로, 마지막 두 Fully-Connected layer는 **1x1 Conv**로 변환
- 입력 이미지의 크기 제약을 없애기 위함



• Single-Scale Evaluation

- Local Response Normalisation은 오히려 성능을 떨어뜨림
- 1×1 conv를 추가한 C보다, 3×3 만 쓴 D가 더 우수해 “공간 컨텍스트” 포착의 중요성 확인.
- 학습 시 **scale jittering**($S \in [256, 512]$)을 도입하면 single-scale 학습 대비 top-1 오류가 27.3→25.5 등 유의미하게 개선.

Table 3: ConvNet performance at a single test scale.

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	25.5	8.0

• Multi-Scale Evaluation

- $Q \in \{S-32, S, S+32\}$ 혹은 $Q \in \{256, 384, 512\}$ 로 여러 크기에서 예측 후 평균.
- D, E configuration에서 best 성능
- single-scale 테스트보다 1% 정도 오류율 감소.

Table 4: ConvNet performance at multiple test scales.

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
B	256	224,256,288	28.2	9.6
C	256	224,256,288	27.7	9.2
	384	352,384,416	27.8	9.2
	[256; 512]	256,384,512	26.3	8.2
D	256	224,256,288	26.6	8.6
	384	352,384,416	26.5	8.6
	[256; 512]	256,384,512	24.8	7.5
E	256	224,256,288	26.9	8.7
	384	352,384,416	26.7	8.6
	[256; 512]	256,384,512	24.8	7.5

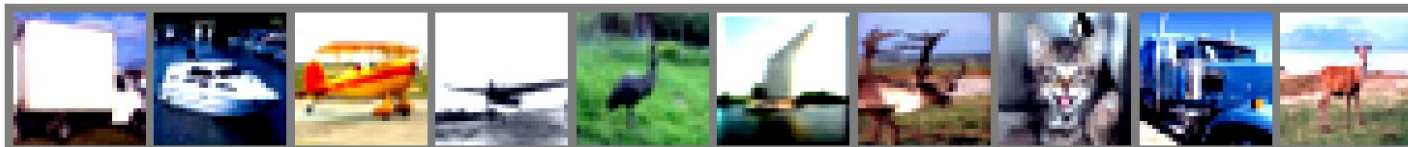
- > 이전 ILSVRC 수상작 모델들에서 **SOTA**를 달성했으며, 같은 해 1위 모델인 GoogLeNet에 비했을때 **양상블 대비 높은 성능**을 보임
- > 전통적인 ConvNet 아키텍처에 깊이만 깊어져도 이미지 분류 **정확도가 크게 향상됨**을 명확히 확인함.
- > **다양한 데이터셋**과 과제에서도 깊은 표현이 복잡한 파이프라인에 견줄 만큼 뛰어난 **일반화 능력**을 발휘함.

Table 7: **Comparison with the state of the art in ILSVRC classification.** Our method is denoted as “VGG”. Only the results obtained without outside training data are reported.

Method	top-1 val. error (%)	top-5 val. error (%)	top-5 test error (%)
VGG (2 nets, multi-crop & dense eval.)	23.7	6.8	6.8
VGG (1 net, multi-crop & dense eval.)	24.4	7.1	7.0
VGG (ILSVRC submission, 7 nets, dense eval.)	24.7	7.5	7.3
GoogLeNet (Szegedy et al., 2014) (1 net)	-	7.9	-
GoogLeNet (Szegedy et al., 2014) (7 nets)	-	6.7	-
MSRA (He et al., 2014) (11 nets)	-	-	8.1
MSRA (He et al., 2014) (1 net)	27.9	9.1	9.1
Clarifai (Russakovsky et al., 2014) (multiple nets)	-	-	11.7
Clarifai (Russakovsky et al., 2014) (1 net)	-	-	12.5
Zeiler & Fergus (Zeiler & Fergus, 2013) (6 nets)	36.0	14.7	14.8
Zeiler & Fergus (Zeiler & Fergus, 2013) (1 net)	37.5	16.0	16.1
OverFeat (Sermanet et al., 2014) (7 nets)	34.0	13.2	13.6
OverFeat (Sermanet et al., 2014) (1 net)	35.7	14.2	-
Krizhevsky et al. (Krizhevsky et al., 2012) (5 nets)	38.1	16.4	16.4
Krizhevsky et al. (Krizhevsky et al., 2012) (1 net)	40.7	18.2	-

VGGNet 구현 실습

1 VGGNet 백본 논문



Labels:
truck ship plane plane bird ship deer cat truck deer

```
def conv_2_block(in_dim, out_dim):  
    return nn.Sequential(  
        nn.Conv2d(in_dim, out_dim, kernel_size=3, padding=1),  
        nn.ReLU(inplace=True),  
        nn.Conv2d(out_dim, out_dim, kernel_size=3, padding=1),  
        nn.ReLU(inplace=True),  
        nn.MaxPool2d(2, 2)  
    )
```

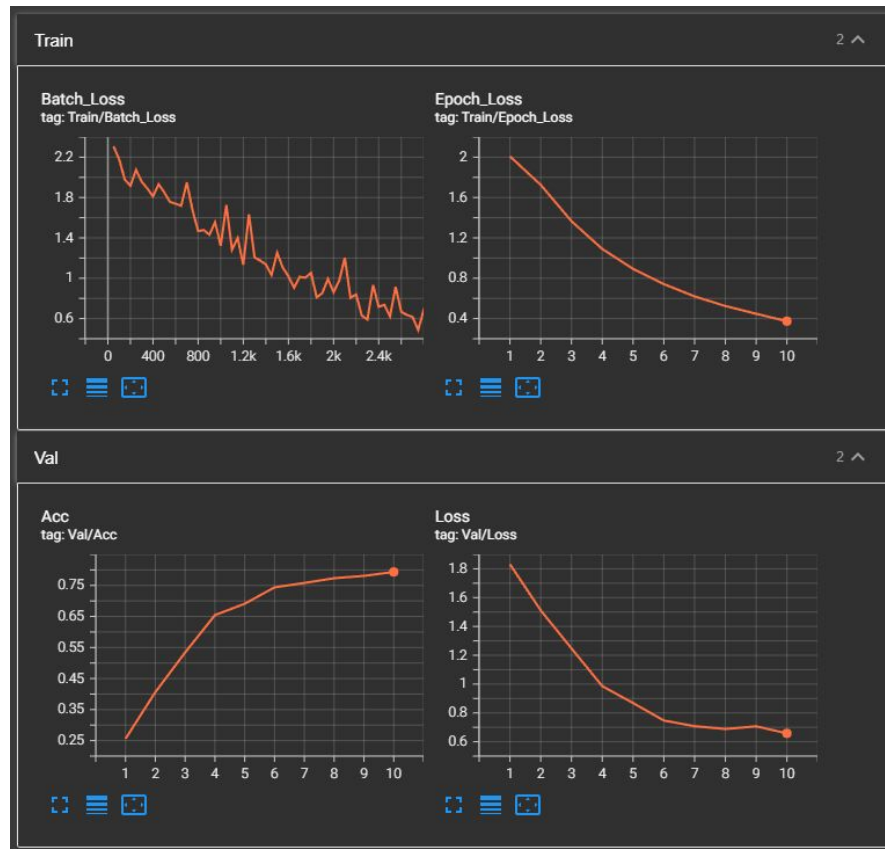
```
def conv_3_block(in_dim, out_dim):  
    return nn.Sequential(  
        nn.Conv2d(in_dim, out_dim, 3, padding=1),  
        nn.ReLU(inplace=True),  
        nn.Conv2d(out_dim, out_dim, 3, padding=1),  
        nn.ReLU(inplace=True),  
        nn.Conv2d(out_dim, out_dim, 3, padding=1),  
        nn.ReLU(inplace=True),  
        nn.MaxPool2d(2, 2)  
    )
```

```
batch_size = 256  
learning_rate = 0.01  
num_epochs = 10
```

```
model = VGG16_CIFAR(base_dim=64, num_classes=10).to(device)  
criterion = nn.CrossEntropyLoss()  
optimizer = optim.SGD(model.parameters(),  
                        lr=learning_rate,  
                        momentum=0.9,  
                        weight_decay=5e-4)
```

```
class VGG16_CIFAR(nn.Module):  
    def __init__(self, base_dim=64, num_classes=10):  
        super().__init__()  
        self.features = nn.Sequential(  
            conv_2_block(3, base_dim),  
            conv_2_block(base_dim, base_dim*2),  
            conv_3_block(base_dim*2, base_dim*4),  
            conv_3_block(base_dim*4, base_dim*8),  
            conv_3_block(base_dim*8, base_dim*8),  
        )  
        self.classifier = nn.Sequential(  
            nn.Linear(base_dim*8*1*1, 4096),  
            nn.ReLU(inplace=True),  
            nn.Dropout(0.5),  
            nn.Linear(4096, 1000),  
            nn.ReLU(inplace=True),  
            nn.Dropout(0.5),  
            nn.Linear(1000, num_classes),  
        )
```

```
def forward(self, x):  
    x = self.features(x)  
    x = x.view(x.size(0), -1)  
    return self.classifier(x)
```



>>> Final Test Accuracy: 0.7900

2021 IEEE 3rd International Conference on Communications, Information System and Computer Engineering (CISCE 2021)

Classifying Bone Metastasized SPECT Images in RGB Format with VGGNets

Xiaoyan Chen^{1,3*}, Ziwen Zheng^{1,3}, Liangxia Liu^{1,3}, Weili Wang^{1,3}, Qiang Lin^{1,2,3*}

¹Key Laboratory of China's Ethnic Languages and Information Technology of Ministry of Education, Northwest Minzu University, Lanzhou, Gansu 730000, China

²School of Mathematics and Computer Science, Northwest Minzu University, Lanzhou, Gansu, 730030, China;

³Key Laboratory of Streaming Data Computing and Applications, Northwest Minzu University, Lanzhou 730030, Gansu, China.*931343519@qq.com, qianglin@xbmu.edu.cn

Abstract—SPECT imaging is widely used for displaying lesions of bone metastasis in nuclear medicine. With SPECT images in RGB format, in this work, we propose to develop automated classification models based on VGG framework, which can answer that whether or not an image contain lesions. Experimental evaluation conducted on a set of real-world data of SPECT imaging shows that the **VGG13** model can effectively **classify bone metastasis SPECT images**, obtaining a value of **0.992**, 0.99, 0.99 and 0.95 for **accuracy Acc**, precision Pre, recall Rec and F-1 scores, respectively.

good results when used in traditional image classification [13]. Therefore, this experiment selects the VGG series models as the basic model to construct a reliable classifier for SPECT images.

The structure of this article is as follows: Section II introduces the source of experimental data and preprocessing. Section III presents several VGG-based models used in this experiment. Section IV reports the experimental. And the fifth part summarizes and prospects.

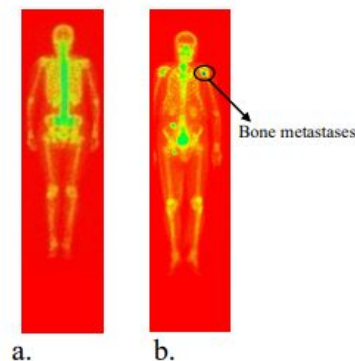


Figure 1 Example of bone metastasis in SPECT nuclear medicine image

(a. Original image b. Bone metastasis)

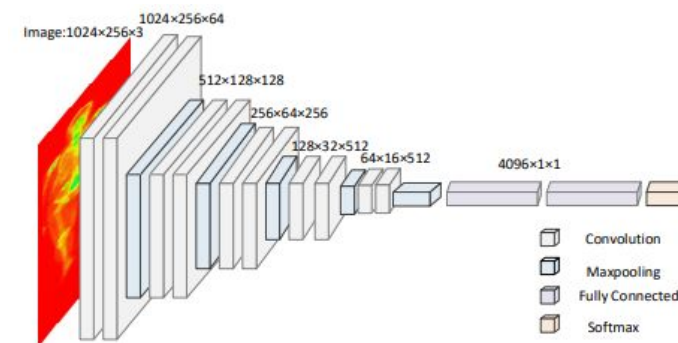


Figure 4 VGG13 model structure diagram

- > 병원 내부 환자들의 RGB 형식 ***SPECT** 데이터 사용 (오픈X)
암세포의 **골전이**를 확인하기 위해 전신 뼈를 촬영한 DICOM 데이터
 - Bone metastasis (312장), Normal (330장)

- > 데이터가 제한적이므로 rotation과 translation을 통해 augmentation

***SPECT**(Single Photon Emission Computed Tomography)

: 방사선 의약품을 사용해 신체 내부의 기능적 정보를 얻는 핵의학 영상 검사

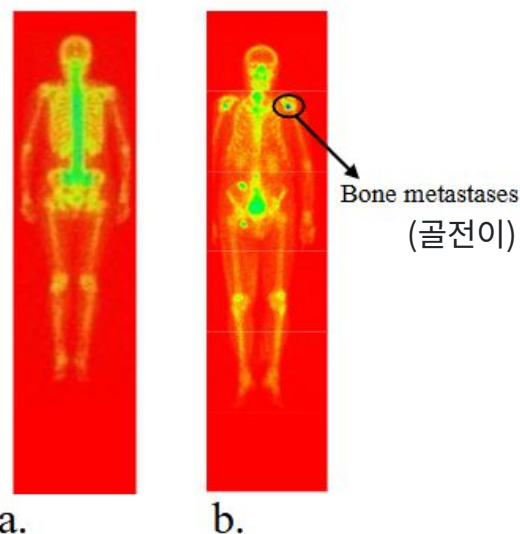


Figure 1 Example of bone metastasis in SPECT nuclear medicine image
(a. Original image b. Bone metastasis)

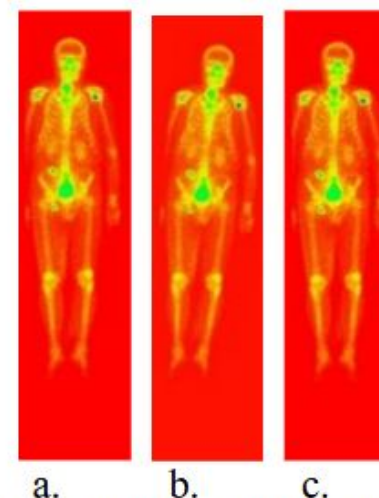


Figure 3 Examples of rotation and translation
(a. Original image; b. Rotate 5° clockwise; c. Pan 2° to the right)

Bone metastasis : 312 → 2,184
Normal : 330 → 2,310

- > VGG11, VGG13, VGG16 각각 적용 후 Classification 성능 비교
- > 레이어가 깊지 않은 모델인 **VGG13**에서 오히려 높은 성능을 보임
- > 데이터셋에 따라 올바른 모델을 선택해야하고, 단순히 depth를 깊게 쌓기만 하는 기법엔 한계가 있는 것으로 보임

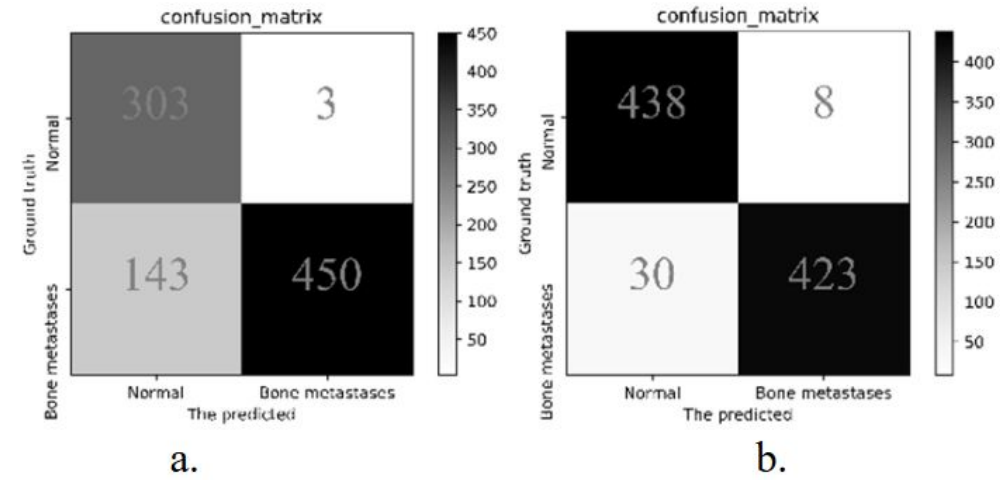
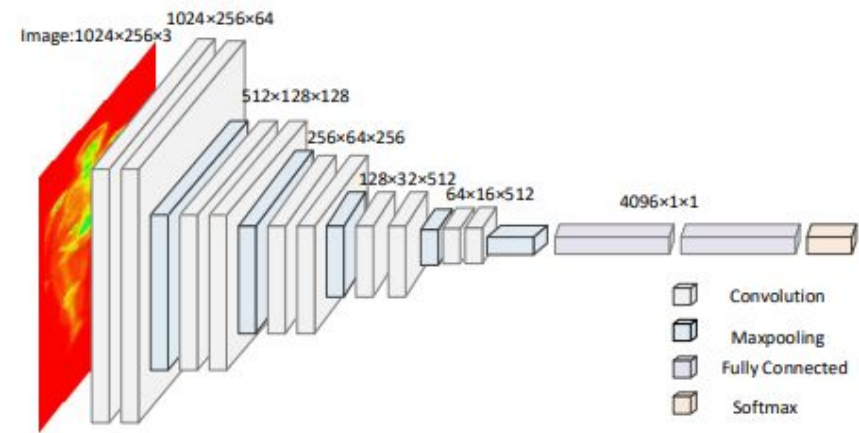


Figure 6 The confusion matrix of the expanded data set under the models VGG11 and VGG13

(a.VGG11's confusion matrix; b.VGG13's confusion matrix)

Table 3 Performance of VGG model on different data sets

	Original			Extended		
	VGG11	VGG13	VGG16	VGG11	VGG13	VGG16
Pre	0.53	0.53	0.55	0.97	0.99	0.53
Rec	1.00	1.00	1.00	0.99	0.99	1.00
F-1	0.70	0.70	0.71	0.76	0.95	0.70

QnA